

# Tutorato: Intel x86-64 e ARM

## Esercizio 1

Tradurre la seguente espressione C in Assembly RISC-V, Intel e ARM.

```
int es_1(int b, int c, int d) {  
    return (b + c) - d;  
}
```

## Esercizio 2

Tradurre la seguente istruzione di accesso in memoria ad un array di `int`.

```
void es_2(int* A, int h) {  
    A[4] = h + A[2];  
}
```

## Esercizio 3

Tradurre la seguente operazione sfruttando in modo efficiente le operazioni logiche.

```
int es_3(int b) {  
    return b * 8;  
}
```

## Esercizio 4

Tradurre il seguente blocco condizionale `if-else`.

```
int es_4(int i, int j, int g, int h) {  
    int f;  
    if (i < j) f = g + h;  
    else f = g - h;  
    return f;  
}
```

## Esercizio 5

Tradurre il seguente semplice costrutto if con incremento.

```
int es_5(int i, int j, int f) {
    if (i == j) f = f + 1;
    return f;
}
```

## Esercizio 6

Tradurre il seguente blocco if-else di assegnazione.

```
int es_6(int i, int j, int g, int h) {
    int f;
    if (i < j) f = g;
    else f = h;
    return f;
}
```

## Esercizio 7

Tradurre la seguente operazione condizionale (!=).

```
int es_7(int i, int j) {
    if (i != j) i = i + 5;
    return i;
}
```

## Esercizio 8

Tradurre il seguente controllo limite (clamp a 0).

```
int es_8(int b, int c) {
    int a = b + c;
    if (a < 0) a = 0;
    return a;
}
```

## Esercizio 9

Tradurre il seguente ciclo while in assembly.

```
int es_9(int* salva, int i, int k) {
    while (salva[i] == k) {
        i += 1;
    }
    return i;
}
```